

The Addison-Wesley Signature Series



A MARTIN FOWLER SIGNATURE
BOOK
Martin

CONTINUOUS DELIVERY

RELIABLE SOFTWARE RELEASES THROUGH BUILD,
TEST, AND DEPLOYMENT AUTOMATION

JEZ HUMBLE
DAVID FARLEY



Foreword by Martin Fowler

Praise for *Continuous Delivery*

“If you need to deploy software more frequently, this book is for you. Applying it will help you reduce risk, eliminate tedious work, and increase confidence. I’ll be using the principles and practices here on all my current projects.”

—*Kent Beck, Three Rivers Institute*

“Whether or not your software development team already understands that continuous integration is every bit as necessary as source code control, this is required reading. This book is unique in tying the whole development and delivery process together, providing a philosophy and principles, not just techniques and tools. The authors make topics from test automation to automated deployment accessible to a wide audience. Everyone on a development team, including programmers, testers, system administrators, DBAs, and managers, needs to read this book.”

—*Lisa Crispin, co-author of Agile Testing*

“For many organizations Continuous Delivery isn’t just a deployment methodology, it’s critical to doing business. This book shows you how to make Continuous Delivery an effective reality in your environment.”

—*James Turnbull, author of Pulling Strings with Puppet*

“A clear, precise, well-written book that gives readers an idea of what to expect for the release process. The authors give a step-by-step account of expectations and hurdles for software deployment. This book is a necessity for any software engineer’s library.”

—*Leyna Cotran, Institute for Software Research, University of California, Irvine*

“Humble and Farley illustrates what makes fast-growing web applications successful. Continuous deployment and delivery has gone from controversial to commonplace and this book covers it excellently. It’s truly the intersection of development and operations on many levels, and these guys nailed it.”

—*John Allspaw, VP Technical Operations, Etsy.com and author of The Art of Capacity Planning and Web Operations*

“If you are in the business of building and delivering a software-based service, you would be well served to internalize the concepts that are so clearly explained in Continuous Delivery. But going beyond just the concepts, Humble and Farley provide an excellent playbook for rapidly and reliably delivering change.”

—*Damon Edwards, President of DTO Solutions and co-editor of dev2ops.org*

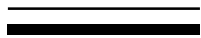
“I believe that anyone who deals with software releases would be able to pick up this book, go to any chapter and quickly get valuable information; or read the book from cover to cover and be able to streamline their build and deploy process in a way that makes sense for their organization. In my opinion, this is an essential handbook for building, deploying, testing, and releasing software.”

—*Sarah Edrie, Director of Quality Engineering, Harvard Business School*

“Continuous Delivery is the logical next step after Continuous Integration for any modern software team. This book takes the admittedly ambitious goal of constantly delivering valuable software to customers, and makes it achievable through a set of clear, effective principles and practices.”

—*Rob Sanheim, Principal at Relevance, Inc.*

This page intentionally left blank



Continuous Delivery

This page intentionally left blank

Continuous Delivery

Jez Humble and David Farley

◆ Addison-Wesley

Upper Saddle River, NJ • Boston • Indianapolis • San Francisco
New York • Toronto • Montreal • London • Munich • Paris • Madrid
Cape Town • Sydney • Tokyo • Singapore • Mexico City

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The authors and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

The publisher offers excellent discounts on this book when ordered in quantity for bulk purchases or special sales, which may include electronic versions and/or custom covers and content particular to your business, training goals, marketing focus, and branding interests. For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the United States, please contact intlcs@pearson.com.

Visit us on the Web: informit.com/aw

Library of Congress Cataloging-in-Publication Data:

Humble, Jez.

Continuous delivery : reliable software releases through build, test, and deployment automation
/ Jez Humble, David Farley.
p. cm.

Includes bibliographical references and index.

ISBN 978-0-321-60191-9 (hardback : alk. paper) 1. Computer software--Development.
2. Computer software--Reliability. 3. Computer software--Testing. I. Farley, David, 1959-
II. Title.

QA76.76.D47H843 2010

005.1--dc22

2010022186

Copyright © 2011 Pearson Education, Inc.

All rights reserved. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearson.com/permissions/.

ISBN-13: 978-0-321-60191-9

ISBN-10: 0-321-60191-2

This book is dedicated to my dad, who has always given me his unconditional love and support.

—Jez

This book is dedicated to my dad, who always showed me the right direction.

—Dave

This page intentionally left blank

Contents

Foreword	xxi
Preface	xxiii
Acknowledgments	xxxix
About the Authors	xxxix
Part I: Foundations	1
Chapter 1: The Problem of Delivering Software	3
Introduction	3
Some Common Release Antipatterns	4
<i>Antipattern: Deploying Software Manually</i>	5
<i>Antipattern: Deploying to a Production-like Environment Only after Development Is Complete</i>	7
<i>Antipattern: Manual Configuration Management of Production Environments</i>	9
<i>Can We Do Better?</i>	10
How Do We Achieve Our Goal?	11
<i>Every Change Should Trigger the Feedback Process</i>	13
<i>The Feedback Must Be Received as Soon as Possible</i>	14
<i>The Delivery Team Must Receive Feedback and Then Act on It</i>	15
<i>Does This Process Scale?</i>	16
What Are the Benefits?	17
<i>Empowering Teams</i>	17
<i>Reducing Errors</i>	18
<i>Lowering Stress</i>	20
<i>Deployment Flexibility</i>	21
<i>Practice Makes Perfect</i>	22

The Release Candidate	22
<i>Every Check-in Leads to a Potential Release</i>	23
Principles of Software Delivery	24
<i>Create a Repeatable, Reliable Process for Releasing Software</i>	24
<i>Automate Almost Everything</i>	25
<i>Keep Everything in Version Control</i>	26
<i>If It Hurts, Do It More Frequently, and Bring the Pain Forward</i>	26
<i>Build Quality In</i>	27
<i>Done Means Released</i>	27
<i>Everybody Is Responsible for the Delivery Process</i>	28
<i>Continuous Improvement</i>	28
Summary	29
Chapter 2: Configuration Management	31
Introduction	31
Using Version Control	32
<i>Keep Absolutely Everything in Version Control</i>	33
<i>Check In Regularly to Trunk</i>	35
<i>Use Meaningful Commit Messages</i>	37
Managing Dependencies	38
<i>Managing External Libraries</i>	38
<i>Managing Components</i>	39
Managing Software Configuration	39
<i>Configuration and Flexibility</i>	40
<i>Types of Configuration</i>	41
<i>Managing Application Configuration</i>	43
<i>Managing Configuration across Applications</i>	47
<i>Principles of Managing Application Configuration</i>	47
Managing Your Environments	49
<i>Tools to Manage Environments</i>	53
<i>Managing the Change Process</i>	53
Summary	54
Chapter 3: Continuous Integration	55
Introduction	55
Implementing Continuous Integration	56
<i>What You Need Before You Start</i>	56
<i>A Basic Continuous Integration System</i>	57

Prerequisites for Continuous Integration	59
<i>Check In Regularly</i>	59
<i>Create a Comprehensive Automated Test Suite</i>	60
<i>Keep the Build and Test Process Short</i>	60
<i>Managing Your Development Workspace</i>	62
Using Continuous Integration Software	63
<i>Basic Operation</i>	63
<i>Bells and Whistles</i>	63
Essential Practices	66
<i>Don't Check In on a Broken Build</i>	66
<i>Always Run All Commit Tests Locally before Committing, or Get Your CI Server to Do It for You</i>	66
<i>Wait for Commit Tests to Pass before Moving On</i>	67
<i>Never Go Home on a Broken Build</i>	68
<i>Always Be Prepared to Revert to the Previous Revision</i>	69
<i>Time-Box Fixing before Reverting</i>	70
<i>Don't Comment Out Failing Tests</i>	70
<i>Take Responsibility for All Breakages That Result from Your Changes</i> .	70
<i>Test-Driven Development</i>	71
Suggested Practices	71
<i>Extreme Programming (XP) Development Practices</i>	71
<i>Failing a Build for Architectural Breaches</i>	72
<i>Failing the Build for Slow Tests</i>	73
<i>Failing the Build for Warnings and Code Style Breaches</i>	73
Distributed Teams	75
<i>The Impact on Process</i>	75
<i>Centralized Continuous Integration</i>	76
<i>Technical Issues</i>	76
<i>Alternative Approaches</i>	77
Distributed Version Control Systems	79
Summary	82
Chapter 4: Implementing a Testing Strategy	83
Introduction	83
Types of Tests	84
<i>Business-Facing Tests That Support the Development Process</i>	85
<i>Technology-Facing Tests That Support the Development Process</i>	89
<i>Business-Facing Tests That Critique the Project</i>	89

<i>Technology-Facing Tests That Critique the Project</i>	91
<i>Test Doubles</i>	91
Real-Life Situations and Strategies	92
<i>New Projects</i>	92
<i>Midproject</i>	94
<i>Legacy Systems</i>	95
<i>Integration Testing</i>	96
Process	99
<i>Managing Defect Backlogs</i>	100
Summary	101
Part II: The Deployment Pipeline	103
Chapter 5: Anatomy of the Deployment Pipeline	105
Introduction	105
What Is a Deployment Pipeline?	106
<i>A Basic Deployment Pipeline</i>	111
Deployment Pipeline Practices	113
<i>Only Build Your Binaries Once</i>	113
<i>Deploy the Same Way to Every Environment</i>	115
<i>Smoke-Test Your Deployments</i>	117
<i>Deploy into a Copy of Production</i>	117
<i>Each Change Should Propagate through the Pipeline Instantly</i>	118
<i>If Any Part of the Pipeline Fails, Stop the Line</i>	119
The Commit Stage	120
<i>Commit Stage Best Practices</i>	121
The Automated Acceptance Test Gate	122
<i>Automated Acceptance Test Best Practices</i>	124
Subsequent Test Stages	126
<i>Manual Testing</i>	128
<i>Nonfunctional Testing</i>	128
Preparing to Release	128
<i>Automating Deployment and Release</i>	129
<i>Backing Out Changes</i>	131
<i>Building on Success</i>	132
Implementing a Deployment Pipeline	133
<i>Modeling Your Value Stream and Creating a Walking Skeleton</i>	133
<i>Automating the Build and Deployment Process</i>	134

<i>Automating the Unit Tests and Code Analysis</i>	135
<i>Automating Acceptance Tests</i>	136
<i>Evolving Your Pipeline</i>	136
Metrics	137
Summary	140
Chapter 6: Build and Deployment Scripting	143
Introduction	143
An Overview of Build Tools	144
<i>Make</i>	146
<i>Ant</i>	147
<i>NAnt and MSBuild</i>	148
<i>Maven</i>	149
<i>Rake</i>	150
<i>Buildr</i>	151
<i>Psake</i>	151
Principles and Practices of Build and Deployment Scripting	152
<i>Create a Script for Each Stage in Your Deployment Pipeline</i>	152
<i>Use an Appropriate Technology to Deploy Your Application</i>	152
<i>Use the Same Scripts to Deploy to Every Environment</i>	153
<i>Use Your Operating System's Packaging Tools</i>	154
<i>Ensure the Deployment Process Is Idempotent</i>	155
<i>Evolve Your Deployment System Incrementally</i>	157
Project Structure for Applications That Target the JVM	157
<i>Project Layout</i>	157
Deployment Scripting	160
<i>Deploying and Testing Layers</i>	162
<i>Testing Your Environment's Configuration</i>	163
Tips and Tricks	164
<i>Always Use Relative Paths</i>	164
<i>Eliminate Manual Steps</i>	165
<i>Build In Traceability from Binaries to Version Control</i>	165
<i>Don't Check Binaries into Version Control as Part of Your Build</i>	166
<i>Test Targets Should Not Fail the Build</i>	166
<i>Constrain Your Application with Integrated Smoke Tests</i>	167
<i>.NET Tips and Tricks</i>	167
Summary	168

Chapter 7: The Commit Stage	169
Introduction	169
Commit Stage Principles and Practices	170
<i>Provide Fast, Useful Feedback</i>	171
<i>What Should Break the Commit Stage?</i>	172
<i>Tend the Commit Stage Carefully</i>	172
<i>Give Developers Ownership</i>	173
<i>Use a Build Master for Very Large Teams</i>	174
The Results of the Commit Stage	174
<i>The Artifact Repository</i>	175
Commit Test Suite Principles and Practices	177
<i>Avoid the User Interface</i>	178
<i>Use Dependency Injection</i>	179
<i>Avoid the Database</i>	179
<i>Avoid Asynchrony in Unit Tests</i>	180
<i>Using Test Doubles</i>	180
<i>Minimizing State in Tests</i>	183
<i>Faking Time</i>	184
<i>Brute Force</i>	185
Summary	185
Chapter 8: Automated Acceptance Testing	187
Introduction	187
Why Is Automated Acceptance Testing Essential?	188
<i>How to Create Maintainable Acceptance Test Suites</i>	190
<i>Testing against the GUI</i>	192
Creating Acceptance Tests	193
<i>The Role of Analysts and Testers</i>	193
<i>Analysis on Iterative Projects</i>	193
<i>Acceptance Criteria as Executable Specifications</i>	195
The Application Driver Layer	198
<i>How to Express Your Acceptance Criteria</i>	200
<i>The Window Driver Pattern: Decoupling the Tests from the GUI</i>	201
Implementing Acceptance Tests	204
<i>State in Acceptance Tests</i>	204
<i>Process Boundaries, Encapsulation, and Testing</i>	206
<i>Managing Asynchrony and Timeouts</i>	207
<i>Using Test Doubles</i>	210

The Acceptance Test Stage	213
<i>Keeping Acceptance Tests Green</i>	214
<i>Deployment Tests</i>	217
Acceptance Test Performance	218
<i>Refactor Common Tasks</i>	219
<i>Share Expensive Resources</i>	219
<i>Parallel Testing</i>	220
<i>Using Compute Grids</i>	220
Summary	222
Chapter 9: Testing Nonfunctional Requirements	225
Introduction	225
Managing Nonfunctional Requirements	226
<i>Analyzing Nonfunctional Requirements</i>	227
Programming for Capacity	228
Measuring Capacity	231
<i>How Should Success and Failure Be Defined for Capacity Tests?</i>	232
The Capacity-Testing Environment	234
Automating Capacity Testing	238
<i>Capacity Testing via the User Interface</i>	240
<i>Recording Interactions against a Service or Public API</i>	241
<i>Using Recorded Interaction Templates</i>	241
<i>Using Capacity Test Stubs to Develop Tests</i>	244
Adding Capacity Tests to the Deployment Pipeline	244
Additional Benefits of a Capacity Test System	247
Summary	248
Chapter 10: Deploying and Releasing Applications	249
Introduction	249
Creating a Release Strategy	250
<i>The Release Plan</i>	251
<i>Releasing Products</i>	252
Deploying and Promoting Your Application	253
<i>The First Deployment</i>	253
<i>Modeling Your Release Process and Promoting Builds</i>	254
<i>Promoting Configuration</i>	257
<i>Orchestration</i>	258
<i>Deployments to Staging Environments</i>	258

Rolling Back Deployments and Zero-Downtime Releases	259
<i>Rolling Back by Redeploying the Previous Good Version</i>	260
Zero-Downtime Releases	260
Blue-Green Deployments	261
Canary Releasing	263
Emergency Fixes	265
Continuous Deployment	266
<i>Continuously Releasing User-Installed Software</i>	267
Tips and Tricks	270
<i>The People Who Do the Deployment Should Be Involved in Creating the Deployment Process</i>	270
Log Deployment Activities	271
Don't Delete the Old Files, Move Them	271
Deployment Is the Whole Team's Responsibility	271
Server Applications Should Not Have GUIs	271
Have a Warm-Up Period for a New Deployment	272
Fail Fast	273
Don't Make Changes Directly on the Production Environment	273
Summary	273
Part III: The Delivery Ecosystem	275
Chapter 11: Managing Infrastructure and Environments	277
Introduction	277
Understanding the Needs of the Operations Team	279
Documentation and Auditing	280
Alerts for Abnormal Events	281
IT Service Continuity Planning	282
Use the Technology the Operations Team Is Familiar With	282
Modeling and Managing Infrastructure	283
Controlling Access to Your Infrastructure	285
Making Changes to Infrastructure	287
Managing Server Provisioning and Configuration	288
Provisioning Servers	288
Ongoing Management of Servers	290
Managing the Configuration of Middleware	295
Managing Configuration	296
Research the Product	298
Examine How Your Middleware Handles State	298

<i>Look for a Configuration API</i>	299
<i>Use a Better Technology</i>	299
Managing Infrastructure Services	300
<i>Multihomed Systems</i>	301
Virtualization	303
<i>Managing Virtual Environments</i>	305
<i>Virtual Environments and the Deployment Pipeline</i>	308
<i>Highly Parallel Testing with Virtual Environments</i>	310
Cloud Computing	312
<i>Infrastructure in the Cloud</i>	313
<i>Platforms in the Cloud</i>	314
<i>One Size Doesn't Have to Fit All</i>	315
<i>Criticisms of Cloud Computing</i>	316
Monitoring Infrastructure and Applications	317
<i>Collecting Data</i>	318
<i>Logging</i>	320
<i>Creating Dashboards</i>	321
<i>Behavior-Driven Monitoring</i>	323
Summary	323
Chapter 12: Managing Data	325
Introduction	325
Database Scripting	326
<i>Initializing Databases</i>	327
Incremental Change	327
<i>Versioning Your Database</i>	328
<i>Managing Orchestrated Changes</i>	329
Rolling Back Databases and Zero-Downtime Releases	331
<i>Rolling Back without Losing Data</i>	331
<i>Decoupling Application Deployment from Database Migration</i>	333
Managing Test Data	334
<i>Faking the Database for Unit Tests</i>	335
<i>Managing the Coupling between Tests and Data</i>	336
<i>Test Isolation</i>	337
<i>Setup and Tear Down</i>	337
<i>Coherent Test Scenarios</i>	337
Data Management and the Deployment Pipeline	338
<i>Data in Commit Stage Tests</i>	338

<i>Data in Acceptance Tests</i>	339
<i>Data in Capacity Tests</i>	341
<i>Data in Other Test Stages</i>	342
Summary	343
Chapter 13: Managing Components and Dependencies	345
Introduction	345
Keeping Your Application Releasable	346
<i>Hide New Functionality Until It Is Finished</i>	347
<i>Make All Changes Incrementally</i>	349
<i>Branch by Abstraction</i>	349
Dependencies	351
<i>Dependency Hell</i>	352
<i>Managing Libraries</i>	354
Components	356
<i>How to Divide a Codebase into Components</i>	356
<i>Pipelining Components</i>	360
<i>The Integration Pipeline</i>	361
Managing Dependency Graphs	363
<i>Building Dependency Graphs</i>	363
<i>Pipelining Dependency Graphs</i>	365
<i>When Should We Trigger Builds?</i>	369
<i>Cautious Optimism</i>	370
<i>Circular Dependencies</i>	372
Managing Binaries	373
<i>How an Artifact Repository Should Work</i>	373
<i>How Your Deployment Pipeline Should Interact with the Artifact Repository</i>	374
Managing Dependencies with Maven	375
<i>Maven Dependency Refactorings</i>	377
Summary	379
Chapter 14: Advanced Version Control	381
Introduction	381
A Brief History of Revision Control	382
CVS	382
Subversion	383
Commercial Version Control Systems	385
Switch Off Pessimistic Locking	386

Branching and Merging	388
<i>Merging</i>	389
<i>Branches, Streams, and Continuous Integration</i>	390
Distributed Version Control Systems	393
<i>What Is a Distributed Version Control System?</i>	393
<i>A Brief History of Distributed Version Control Systems</i>	395
<i>Distributed Version Control Systems in Corporate Environments</i>	396
<i>Using Distributed Version Control Systems</i>	397
Stream-Based Version Control Systems	399
<i>What Is a Stream-Based Version Control System?</i>	399
<i>Development Models with Streams</i>	400
<i>Static and Dynamic Views</i>	403
<i>Continuous Integration with Stream-Based Version Control Systems</i> ...	403
Develop on Mainline	405
<i>Making Complex Changes without Branching</i>	406
Branch for Release	408
Branch by Feature	410
Branch by Team	412
Summary	415
Chapter 15: Managing Continuous Delivery	417
Introduction	417
A Maturity Model for Configuration and Release Management	419
<i>How to Use the Maturity Model</i>	419
Project Lifecycle	421
<i>Identification</i>	422
<i>Inception</i>	423
<i>Initiation</i>	424
<i>Develop and Release</i>	425
<i>Operation</i>	428
A Risk Management Process	429
<i>Risk Management 101</i>	429
<i>Risk Management Timeline</i>	430
<i>How to Do a Risk-Management Exercise</i>	431
Common Delivery Problems—Their Symptoms and Causes	432
<i>Infrequent or Buggy Deployments</i>	433
<i>Poor Application Quality</i>	434
<i>Poorly Managed Continuous Integration Process</i>	435

<i>Poor Configuration Management</i>	436
Compliance and Auditing	436
<i>Automation over Documentation</i>	437
<i>Enforcing Traceability</i>	438
<i>Working in Silos</i>	439
<i>Change Management</i>	440
Summary	442
Bibliography	443
Index	445

Foreword by Martin Fowler

In the late 90s, I paid a visit to Kent Beck, then working in Switzerland for an insurance company. He showed me around his project, and one of the interesting aspects of his highly disciplined team was the fact that they deployed their software into production every night. This regular deployment gave them many advantages: Written software wasn't waiting uselessly until it was deployed, they could respond quickly to problems and opportunities, and the rapid turnaround led to a much deeper relationship between them, their business customer, and their final customers.

In the last decade I've worked at ThoughtWorks, and a common theme of our projects has been reducing the cycle time between an idea and usable software. I see plenty of project stories, and almost all involve a determined shortening of that cycle. While we don't usually do daily deliveries into production, it's now common to see teams doing bi-weekly releases.

Dave and Jez have been part of that sea change, actively involved in projects that have built a culture of frequent, reliable deliveries. They and our colleagues have taken organizations that struggled to deploy software once a year into the world of Continuous Delivery, where releasing becomes routine.

The foundation for the approach, at least for the development team, is Continuous Integration (CI). CI keeps the entire development team in sync, removing the delays due to integration issues. A couple of years ago, Paul Duvall wrote a book on CI in this series. But CI is just the first step. Software that's been successfully integrated into a mainline code stream still isn't software that's out in production doing its job. Dave and Jez's book pick up the story from CI to deal with that "last mile," describing how to build the deployment pipeline that turns integrated code into production software.

This kind of delivery thinking has long been a forgotten corner of software development, falling into a hole between developers and operations teams. So it's no surprise that the techniques in this book rest upon bringing these teams together—a harbinger of the nascent but growing DevOps movement. This process also involves testers, as testing is a key element of ensuring error-free releases.

Threading through all this is a high degree of automation, so things can be done quickly and without error.

Getting all this working takes effort, but benefits are profound. Long, high-intensity releases become a thing of the past. Customers of software see ideas rapidly turn into working code that they can use every day. Perhaps most importantly, we remove one of the biggest sources of baleful stress in software development. Nobody likes those tense weekends trying to get a system upgrade released before Monday dawns.

It seems to me that a book that can show you how to deliver your software frequently and without the usual stresses is a no-brainer to read. For your team's sake, I hope you agree.

Preface

Introduction

Yesterday your boss asked you to demonstrate the great new features of your system to a customer, but you can't show them anything. All your developers are halfway through developing new features and none of them can run the application right now. You have code, it compiles, and all the unit tests pass on your continuous integration server, but it takes a couple of days to release the new version into the publicly accessible UAT environment. Isn't it unreasonable to expect the demo at such short notice?

You have a critical bug in production. It is losing money for your business every day. You know what the fix is: A one-liner in a library that is used in all three layers of your three-tier system, and a corresponding change to one database table. But the last time you released a new version of your software to production it took a weekend of working until 3 A.M., and the person who did the deployment quit in disgust shortly afterward. You know the next release is going to overrun the weekend, which means the application will be down for a period during the business week. If only the business understood our problems.

These problems, although all too common, are not an inevitable outcome of the software development process: They are an indication that something is wrong. Software release should be a fast, repeatable process. These days, many companies are putting out multiple releases in a *day*. This is possible even with large projects with complex codebases. In this book, we will show you how this is done.

Mary and Tom Poppendieck asked, "How long would it take your organization to deploy a change that involves just one single line of code? Do you do this on a repeatable, reliable basis?"¹ The time from deciding that you need to make a change to having it in production is known as the *cycle time*, and it is a vital metric for any project.

1. *Implementing Lean Software Development*, p. 59.

In many organizations, cycle time is measured in weeks or months, and the release process is certainly not repeatable or reliable. It is manual and often requires a team of people to deploy the software even into a testing or staging environment, let alone into production. However, we have come across equally complex projects which started out like this but where, after extensive reengineering, teams were able to achieve a cycle time of hours or even minutes for a critical fix. This was possible because a fully automated, repeatable, reliable process was created for taking changes through the various stages of the build, deploy, test, and release process. Automation is the key. It allows all of the common tasks involved in the creation and deployment of software to be performed by developers, testers, and operations personnel, at the push of a button.

This book describes how to revolutionize software delivery by making the path from idea to realized business value—the cycle time—shorter and safer.

Software delivers no revenue until it is in the hands of its users. This is obvious, but in most organizations the release of software into production is a manually intensive, error-prone, and risky process. While a cycle time measured in months is common, many companies do much worse than this: Release cycles of more than a year are not unknown. For large companies every week of delay between having an idea and releasing the code that implements it can represent millions of dollars in opportunity costs—and yet these are often the ones with the longest cycle times.

Despite all this, the mechanisms and processes that allow for low-risk delivery of software have not become part of the fabric in most of today's software development projects.

Our aim is to make the delivery of software from the hands of developers into production a reliable, predictable, visible, and largely automated process with well-understood, quantifiable risks. Using the approach that we describe in this book, it is possible to go from having an idea to delivering working code that implements it into production in a matter of minutes or hours, while at the same time improving the quality of the software thus delivered.

The vast majority of the cost associated with delivering successful software is incurred after the first release. This is the cost of support, maintenance, adding new features, and fixing defects. This is especially true of software delivered via iterative processes, where the first release contains the minimum amount of functionality providing value to the customer. Hence the title of this book, *Continuous Delivery*, which is taken from the first principle of the Agile Manifesto: "Our highest priority is to satisfy the customer through early and continuous delivery of valuable software" [bibNp0]. This reflects the reality: For successful software, the first release is just the beginning of the delivery process.

All the techniques we describe in this book reduce the time and risks associated with delivering new versions of your software to users. They do this by increasing feedback and improving collaboration between the development, testing, and operations personnel responsible for delivery. These techniques ensure that when you need to modify applications, either to fix bugs or deliver new features, the

time between making modifications and having the results deployed and in use is as low as possible, problems are found early when they are easy to fix, and associated risks are well understood.

Who Is This Book for, and What Does It Cover?

One of the major aims of this book is to improve collaboration between the people responsible for delivering software. In particular, we have in mind developers, testers, systems and database administrators, and managers.

We cover topics from traditional configuration management, source code control, release planning, auditing, compliance, and integration to the automation of your building, testing, and deployment processes. We also describe techniques such as automated acceptance testing, dependency management, database migration, and the creation and management of testing and production environments.

Many people involved in creating software consider these activities secondary to writing code. However, in our experience they take up a great deal of time and effort, and are critical to successful software delivery. When the risks surrounding these activities are not managed adequately, they can end up costing a lot of money, often more than the cost of building the software in the first place. This book provides the information that you need to understand these risks and, more importantly, describes strategies to mitigate them.

This is an ambitious aim, and of course we can't cover all these topics in detail in one book. Indeed we run the risk of alienating each of our target audiences: developers, by failing to treat topics such as architecture, behavior-driven development, and refactoring in depth; testers, by not spending sufficient time on exploratory testing and test management strategies; operations personnel, by not paying due attention to capacity planning, database migration, and production monitoring.

However, books exist that address each of these topics in detail. What we think is lacking in the literature is a book that discusses how all the moving parts fit together: configuration management, automated testing, continuous integration and deployment, data management, environment management, and release management. One of the things that the lean software development movement teaches is that it is important to optimize the whole. In order to do this, a holistic approach is necessary that ties together every part of the delivery process and everybody involved in it. Only when you have control over the progression of every change from introduction to release can you begin to optimize and improve the quality and speed of software delivery.

Our aim is to present a holistic approach, as well as the principles involved in this approach. We will provide you with the information that you will need to decide how to apply these practices in your own projects. We do not believe that there is a "one size fits all" approach to any aspect of software development, let alone a subject area as large as the configuration management and operational control of an enterprise system. However, the fundamentals that we describe in

this book are widely applicable to all sorts of different software projects—big, small, highly technical or short sprints to early value.

As you begin to put these principles into practice, you will discover the areas where more detail is required for your particular situation. There is a bibliography at the end of this book, as well as pointers to other resources online where you can find more information on each of the topics that we cover.

This book consists of three parts. The first part presents the principles behind continuous delivery and the practices necessary to support it. Part two describes the central paradigm of the book—a pattern we call the deployment pipeline. The third part goes into more detail on the ecosystem that supports the deployment pipeline—techniques to enable incremental development; advanced version control patterns; infrastructure, environment and data management; and governance.

Many of these techniques may appear to apply only to large-scale applications. While it is true that much of our experience is with large applications, we believe that even the smallest projects can benefit from a thorough grounding in these techniques, for the simple reason that projects grow. The decisions that you make when starting a small project will have an inevitable impact on its evolution, and by starting off in the right way, you will save yourself (or those who come after you) a great deal of pain further down the line.

Your authors share a background in lean and iterative software development philosophies. By this we mean that we aim to deliver valuable, working software to users rapidly and iteratively, working continuously to remove waste from the delivery process. Many of the principles and techniques that we describe were first developed in the context of large agile projects. However, the techniques that we present in this book are of general applicability. Much of our focus is on improving collaboration through better visibility and faster feedback. This will have a positive impact on every project, whether or not it uses iterative software development processes.

We have tried to ensure that chapters and even sections can be read in isolation. At the very least, we hope that anything you need to know, as well as references to further information, are clearly sign-posted and accessible so that you can use this book as a reference.

We should mention that we don't aim for academic rigor in our treatment of the subjects covered. There are plenty of more theoretical books on the market, many of which provide interesting reading and insights. In particular, we will not spend much time on standards, concentrating instead on battle-tested skills and techniques every person working on a software project will find useful, and explaining them clearly and simply so that they can be used every day in the real world. Where appropriate, we will provide some war stories illustrating these techniques to help place them in context.

Conspectus

We recognize that not everyone will want to read this book from end to end. We have written it so that once you have covered the introduction, you can attack it in several different ways. This has involved a certain amount of repetition, but hopefully not at a level that becomes tedious if you do decide to read it cover-to-cover.

This book consists of three parts. The first part, Chapters 1 to 4, takes you through the basic principles of regular, repeatable, low-risk releases and the practices that support them. Part two, Chapters 5 through 10, describe the deployment pipeline. From Chapter 11 we dive into the ecosystem that supports continuous delivery.

We recommend that everybody read Chapter 1. We believe that people who are new to the process of releasing software, even experienced developers, will find plenty of material challenging their view of what it means to do professional software development. The rest of the book can be dipped into either at your leisure—or when in a panic.

Part I—Foundations

Part I describes the prerequisites for understanding the deployment pipeline. Each chapter builds upon the last.

Chapter 1, “The Problem of Delivering Software,” starts by describing some common antipatterns that we see in many software development teams, and moves on to describe our goal and how to realize it. We conclude by setting out the principles of software delivery upon which the rest of the book is based.

Chapter 2, “Configuration Management,” sets out how to manage everything required to build, deploy, test, and release your application, from source code and build scripts to your environment and application configuration.

Chapter 3, “Continuous Integration,” covers the practice of building and running automated tests against every change you make to your application so you can ensure that your software is always in a working state.

Chapter 4, “Implementing a Testing Strategy,” introduces the various kinds of manual and automated testing that form an integral part of every project, and discusses how to decide which strategy is appropriate for your project.

Part II—The Deployment Pipeline

The second part of the book covers the deployment pipeline in detail, including how to implement the various stages in the pipeline.

Chapter 5, “Anatomy of the Deployment Pipeline,” discusses the pattern that forms the core of this book—an automated process for taking every change from check-in to release. We also discuss how to implement pipelines at both the team and organizational levels.

Chapter 6, “Build and Deployment Scripting,” discusses scripting technologies that can be used for creating automated build and deployment processes, and the best practices for using them.

Chapter 7, “The Commit Stage,” covers the first stage of the pipeline, a set of automated processes that should be triggered the moment any change is introduced into your application. We also discuss how to create a fast, effective commit test suite.

Chapter 8, “Automated Acceptance Testing,” presents automated acceptance testing, from analysis to implementation. We discuss why acceptance tests are essential to continuous delivery, and how to create a cost-effective acceptance test suite that will protect your application’s valuable functionality.

Chapter 9, “Testing Nonfunctional Requirements,” discusses nonfunctional requirements, with an emphasis on capacity testing. We describe how to create capacity tests, and how to set up a capacity testing environment.

Chapter 10, “Deploying and Releasing Applications,” covers what happens after automated testing: push-button promotion of release candidates to manual testing environments, UAT, staging, and finally release, taking in essential topics such as continuous deployment, roll backs, and zero-downtime releases.

Part III—The Delivery Ecosystem

The final part of the book discusses crosscutting practices and techniques that support the deployment pipeline.

Chapter 11, “Managing Infrastructure and Environments,” covers the automated creation, management, and monitoring of environments, including the use of virtualization and cloud computing.

Chapter 12, “Managing Data,” shows how to create and migrate testing and production data through the lifecycle of your application.

Chapter 13, “Managing Components and Dependencies,” starts with a discussion of how to keep your application in a releasable state at all times without branching. We then describe how to organize your application as a collection of components, and how to manage building and testing them.

Chapter 14, “Advanced Version Control,” gives an overview of the most popular tools, and goes into detail on the various patterns for using version control.

Chapter 15, “Managing Continuous Delivery,” sets out approaches to risk management and compliance, and provides a maturity model for configuration and release management. Along the way, we discuss the value of continuous delivery to the business, and the lifecycle of iterative projects that deliver incrementally.

Web Links in This Book

Rather than putting in complete links to external websites, we have shortened them and put in the key in this format: [bibNp0]. You can go to the link in one of two ways. Either use bit.ly, in which case the url for the example key would be <http://bit.ly/bibNp0>. Alternatively, you can use a url shortening service we've installed at <http://continuousdelivery.com/go/> which uses the same keys—so the url for the example key is <http://continuousdelivery.com/go/bibNp0>. The idea is that if for some reason bit.ly goes under, the links are preserved. If the web pages change address, we'll try to keep the shortening service at <http://continuousdelivery.com/go/> up-to-date, so try that if the links don't work at bit.ly.

About the Cover

All books in Martin Fowler's Signature Series have a bridge on the cover. We'd originally planned to use a photo of the Iron Bridge, but it had already been chosen for another book in the series. So instead, we chose another British bridge: the Forth Railway Bridge, captured here in a stunning photo by Stewart Hardy.

The Forth Railway Bridge was the first bridge in the UK constructed using steel, manufactured using the new Siemens-Martin open-hearth process, and delivered from two steel works in Scotland and one in Wales. The steel was delivered in the form of manufactured tubular trusses—the first time a bridge in the UK used mass-produced parts. Unlike earlier bridges, the designers, Sir John Fowler, Sir Benjamin Baker, and Allan Stewart, made calculations for incidence of erection stresses, provisions for reducing future maintenance costs, and calculations for wind pressures and the effect of temperature stresses on the structure—much like the functional and nonfunctional requirements we make in software. They also supervised the construction of the bridge to ensure these requirements were met.

The bridge's construction involved more than 4,600 workers, of whom tragically around one hundred died and hundreds more were crippled. However, the end result is one of the marvels of the industrial revolution: At the time of completion in 1890 it was the longest bridge in the world, and at the start of the 21st century it remains the world's second longest cantilever bridge. Like a long-lived software project, the bridge needs constant maintenance. This was planned for as part of the design, with ancillary works for the bridge including not only a maintenance workshop and yard but a railway "colony" of some fifty houses at Dalmeny Station. The remaining working life of the bridge is estimated at over 100 years.

Colophon

This book was written directly in DocBook. Dave edited the text in TextMate, and Jez used Aquamacs Emacs. The diagrams were created with OmniGraffle. Dave and Jez were usually not in the same part of the world, and collaborated by having everything checked in to Subversion. We also employed continuous integration, using a CruiseControl.rb server that ran dblatex to produce a PDF of the book every time one of us committed a change.

A month before the book went to print, Dmitry Kirsanov and Alina Kirsanova started the production work, collaborating with the authors through their Subversion repository, email, and a shared Google Docs table for coordination. Dmitry worked on copyediting of the DocBook source in XEmacs, and Alina did everything else: typesetting the pages using a custom XSLT stylesheet and an XSL-FO formatter, compiling and editing the Index from the author's indexing tags in the source, and final proofreading of the book.

Acknowledgments

Many people have contributed to this book. In particular, we'd like to thank our reviewers: David Clack, Leyna Cotran, Lisa Crispin, Sarah Edrie, Damon Edwards, Martin Fowler, James Kovacs, Bob Maksimchuk, Elliotte Rusty Harold, Rob Sanheim, and Chris Smith. We'd also like to extend special thanks to our editorial and production team at Addison-Wesley: Chris Guzikowski, Raina Chrobak, Susan Zahn, Kristy Hart, and Andy Beaster. Dmitry Kirsanov and Alina Kirsanova did a fantastic job of copyediting and proofreading the book, and typesetting it using their fully automated system.

Many of our colleagues have been instrumental in developing the ideas in this book, including (in no particular order) Chris Read, Sam Newman, Dan North, Dan Worthington-Bodart, Manish Kumar, Kraig Parkinson, Julian Simpson, Paul Julius, Marco Jansen, Jeffrey Fredrick, Ajey Gore, Chris Turner, Paul Hammant, Hu Kai, Qiao Yandong, Qiao Liang, Derek Yang, Julias Shaw, Deepthi, Mark Chang, Dante Briones, Li Guanglei, Erik Doernenburg, Kraig Parkinson, Ram Narayanan, Mark Rickmeier, Chris Stevenson, Jay Flowers, Jason Sankey, Daniel Ostermeier, Rolf Russell, Jon Tirsén, Timothy Reaves, Ben Wyeth, Tim Harding, Tim Brown, Pavan Kadambi Sudarshan, Stephen Foresheew, Yogi Kulkarni, David Rice, Chad Wathington, Jonny LeRoy, and Chris Briesemeister.

Jez would like to thank his wife, Rani, for being the most loving partner he could wish for, and for cheering him up when he was grumpy during the writing of this book. He also thanks his daughter, Amrita, for her babbling, cuddles, and big gummy smiles. He is also profoundly grateful to his colleagues at ThoughtWorks for making it such an inspiring place to work, and to Cyndi Mitchell and Martin Fowler for their support of this book. Finally, a big shout out to Jeffrey Fredrick and Paul Julius for creating CITCON, and to the people he met there for many great conversations.

Dave would like to thank his wife Kate, and children Tom and Ben, for their unfailing support at every point, in this project and in many others. He would also like to make a special mention of ThoughtWorks, who, although no longer his employer, provided an environment of enlightenment and encouragement for

the people that worked there, thus fostering a creative approach to finding solutions, many of which populate the pages of this book. In addition, he would like to thank his current employer, LMAX, with a special mention for Martin Thompson, for their support, trust, and willing adoption of the techniques described in this book in an intensely challenging technical environment of world-class high-performance computing.

About the Authors

Jez Humble has been fascinated by computers and electronics since getting his first ZX Spectrum at age 11, and spent several years hacking on Acorn machines in 6502 and ARM assembler and BASIC until he was old enough to get a proper job. He got into IT in 2000, just in time for the dot-com bust. Since then he has worked as a developer, system administrator, trainer, consultant, manager, and speaker. He has worked with a variety of platforms and technologies, consulting for nonprofits, telecoms, financial services, and online retail companies. Since 2004 he has worked for ThoughtWorks and ThoughtWorks Studios in Beijing, Bangalore, London, and San Francisco. He holds a BA in Physics and Philosophy from Oxford University and an MMus in Ethnomusicology from the School of Oriental and African Studies, University of London. He is presently living in San Francisco with his wife and daughter.

Dave Farley has been having fun with computers for nearly 30 years. Over that period he has worked on most types of software—from firmware, through tinkering with operating systems and device drivers, to writing games and commercial applications of all shapes and sizes. He started working in large-scale distributed systems about twenty years ago, doing research into the development of loose-coupled, message-based systems—a forerunner of SOA. He has a wide range of experience leading the development of complex software in teams, both large and small, in the UK and USA. Dave was an early adopter of agile development techniques, employing iterative development, continuous integration, and significant levels of automated testing on commercial projects from the early 1990s. He honed his approach to agile development during his four-and-a-half-year stint at ThoughtWorks where he was a technical principal working on some of their biggest and most challenging projects. Dave is currently working for the London Multi-Asset Exchange (LMAX), an organization that is building one of the highest-performance financial exchanges in the world, where they rely upon all of the major techniques described in this book.

This page intentionally left blank

Part I



Foundations

This page intentionally left blank

Chapter 1

The Problem of Delivering Software

Introduction

The most important problem that we face as software professionals is this: If somebody thinks of a good idea, how do we deliver it to users as quickly as possible? This book shows how to solve this problem.

We focus on the build, deploy, test, and release process, about which relatively little has been written. This is not because we think that software development approaches are not important; rather, that without a focus on the other aspects of the software lifecycle—aspects that are all too commonly treated as peripheral to the overall problem—it is impossible to achieve reliable, rapid, low-risk software releases that get the fruits of our labors into the hands of our users in an efficient manner.

There are many software development methodologies, but they focus primarily on requirement management and its impact on the development effort. There are many excellent books that cover in detail different approaches to software design, development, and testing; but these, too, cover only a fragment of the *value stream* that delivers value to the people and organizations that sponsor our efforts.

What happens once requirements are identified, solutions designed, developed, and tested? How are these activities joined together and coordinated to make the process as efficient and reliable as we can make it? How do we enable developers, testers, build and operations personnel to work together effectively?

This book describes an effective pattern for getting software from development to release. We describe techniques and best practices that help to implement this pattern and show how this approach interfaces with other aspects of software delivery.

The pattern that is central to this book is the *deployment pipeline*. A deployment pipeline is, in essence, an automated implementation of your application's build, deploy, test, and release process. Every organization will have differences in the implementation of their deployment pipelines, depending on their value

stream for releasing software, but the principles that govern them do not vary. An example of a deployment pipeline is given in Figure 1.1.



Figure 1.1 *The deployment pipeline*

The way the deployment pipeline works, in a paragraph, is as follows. Every change that is made to an application's configuration, source code, environment, or data, triggers the creation of a new instance of the pipeline. One of the first steps in the pipeline is to create binaries and installers. The rest of the pipeline runs a series of tests on the binaries to prove that they can be released. Each test that the release candidate passes gives us more confidence that this particular combination of binary code, configuration information, environment, and data will work. If the release candidate passes all the tests, it can be released.

The deployment pipeline has its foundations in the process of *continuous integration* and is in essence the principle of continuous integration taken to its logical conclusion.

The aim of the deployment pipeline is threefold. First, it makes every part of the process of building, deploying, testing, and releasing software visible to everybody involved, aiding collaboration. Second, it improves feedback so that problems are identified, and so resolved, as early in the process as possible. Finally, it enables teams to deploy and release any version of their software to any environment at will through a fully automated process.

Some Common Release Antipatterns

The day of a software release tends to be a tense one. Why should this be the case? For most projects, it is the degree of risk associated with the process that makes release a scary time.

In many software projects, release is a manually intensive process. The environments that host the software are often crafted individually, usually by an operations or IS team. Third-party software that the application relies on is installed. The software artifacts of the application itself are copied to the production host environments. Configuration information is copied or created through the admin consoles of web servers, applications servers, or other third-party components of the system. Reference data is copied, and finally the application is started, piece by piece if it is a distributed or service-oriented application.

The reason for the nervousness should be clear: There is quite a lot to go wrong in this process. If any step is not perfectly executed, the application won't run

properly. At this point it may not be at all clear where the error is, or which step went wrong.

The rest of this book discusses how to avoid these risks—how to reduce the stress on release days, and how to ensure that each release is predictably reliable.

Before that, let's be clear about the kinds of process failures that we are trying to avoid. Here are a few common antipatterns that prevent a reliable release process, but nevertheless are so common as to be the norm in our industry.

Antipattern: Deploying Software Manually

Most modern applications of any size are complex to deploy, involving many moving parts. Many organizations release software manually. By this we mean that the steps required to deploy such an application are treated as separate and atomic, each performed by an individual or team. Judgments must be made within these steps, leaving them prone to human error. Even if this is not the case, differences in the ordering and timing of these steps can lead to different outcomes. These differences are rarely good.

The signs of this antipattern are:

- The production of extensive, detailed documentation that describes the steps to be taken and the ways in which the steps may go wrong
- Reliance on manual testing to confirm that the application is running correctly
- Frequent calls to the development team to explain why a deployment is going wrong on a release day
- Frequent corrections to the release process during the course of a release
- Environments in a cluster that differ in their configuration, for example application servers with different connection pool settings, filesystems with different layouts, etc.
- Releases that take more than a few minutes to perform
- Releases that are unpredictable in their outcome, that often have to be rolled back or run into unforeseen problems
- Sitting bleary-eyed in front of a monitor at 2 A.M. the day after the release day, trying to figure out how to make it work

Instead . . .

Over time, deployments should tend towards being fully automated. There should be two tasks for a human being to perform to deploy software into a development,

test, or production environment: to pick the version and environment and to press the “deploy” button. Releasing packaged software should involve a single automated process that creates the installer.

We discuss automation a lot in the course of this book, and we know that some people aren’t totally sold on the idea. Let us explain why we see automated deployment as an indispensable goal.

- When deployments aren’t fully automated, errors will occur every time they are performed. The only question is whether or not the errors are significant. Even with excellent deployment tests, bugs can be hard to track down.
- When the deployment process is not automated, it is not repeatable or reliable, leading to time wasted on debugging deployment errors.
- A manual deployment process has to be documented. Maintaining the documentation is a complex and time-consuming task involving collaboration between several people, so the documentation is generally incomplete or out-of-date at any given time. A set of automated deployment scripts serves as documentation, and it will always be up-to-date and complete, or the deployment will not work.
- Automated deployments encourage collaboration, because everything is explicit in a script. Documentation has to make assumptions about the level of knowledge of the reader and in reality is usually written as an aide-memoire for the person performing the deployment, making it opaque to others.
- A corollary of the above: Manual deployments depend on the deployment expert. If he or she is on vacation or quits work, you are in trouble.
- Performing manual deployments is boring and repetitive and yet needs significant degree of expertise. Asking experts to do boring and repetitive, and yet technically demanding tasks is the most certain way of ensuring human error that we can think of, short of sleep deprivation, or inebriation. Automating deployments frees your expensive, highly skilled, overworked staff to work on higher-value activities.
- The only way to test a manual deployment process is to do it. This is often time-consuming and expensive. An automated deployment process is cheap and easy to test.
- We have heard it said that a manual process is more auditable than an automated one. We are completely baffled by this statement. With a manual process, there is no guarantee that the documentation has been followed. Only an automated process is fully auditable. What is more auditable than a working deployment script?

The automated deployment process must be used by everybody, and it should be the only way in which the software is ever deployed. This discipline ensures that the deployment script will work when it is needed. One of the principles that we describe in this book is to use the same script to deploy to every environment. If you use the same script to deploy to every environment, then the deployment-to-production path will have been tested hundreds or even thousands of times before it is needed on release day. If any problems occur upon release, you can be certain they are problems with environment-specific configuration, not your scripts.

We are certain that, occasionally, manually intensive releases work smoothly. We may well have been unlucky in having mostly seen the bad ones. However, if this is not recognized as a potentially error-prone step in the process of software production, why is it attended by such ceremony? Why all the process and documentation? Why are the teams of people brought in during weekends? Why have people waiting on standby in case things go less than well?

Antipattern: Deploying to a Production-like Environment Only after Development Is Complete

In this pattern, the first time the software is deployed to a production-like environment (for example, staging) is once most of the development work is done—at least, “done” as defined by the development team.

The pattern looks a bit like this.

- If testers have been involved in the process up to this point, they have tested the system on development machines.
- Releasing into staging is the first time that operations people interact with the new release. In some organizations, separate operations teams are used to deploy the software into staging and production. In this case, the first time an operations person sees the software is the day it is released into production.
- Either a production-like environment is expensive enough that access to it is strictly controlled, or it is not in place on time, or nobody bothered to create one.
- The development team assembles the correct installers, configuration files, database migrations, and deployment documentation to pass to the people who perform the actual deployment—all of it untested in an environment that looks like production or staging.
- There is little, if any, collaboration between the development team and the people who actually perform deployments to create this collateral.

When the deployment to staging occurs, a team is assembled to perform it. Sometimes this team has all the necessary skills, but often in very large organizations the responsibilities for deployment are divided between several groups. DBAs, middleware teams, web teams, and others all take a hand in deploying the latest version of the application. Since the various steps have never been tested in staging, they often have errors. The documentation misses important steps. The documentation and scripts make assumptions about the version or configuration of the target environment that are wrong, causing the deployment to fail. The deployment team has to guess at the intentions of the development team.

Often the poor collaboration that causes so many problems in deployment to staging is shored up with ad-hoc telephone calls, emails, and quick fixes. A very disciplined team will incorporate all of this communication into the deployment plan—but it is rare for this process to be effective. As pressure increases, the defined process for collaboration between the development and deployment teams is subverted, in order to get the deployment done within the time allocated to the deployment team.

In the process of performing the deployment, it is not uncommon to find that incorrect assumptions about the production environment have been baked into the design of the system. For example, one application we had a hand in deploying used the filesystem to cache data. This worked fine on a developer workstation, but less well in a clustered environment. Solving problems like this one can take a long time, and the application cannot be said to have been deployed until they are resolved.

Once the application is deployed into staging, it is common for new bugs to be found. Unfortunately, there is often no time to fix them all because the deadline is fast approaching and, at this stage of the project, deferring the release date is unacceptable. So the most critical bugs are hurriedly patched up, and a list of known defects is stored by the project manager for safekeeping, to be deprioritized when work begins on the next release.

Sometimes it can be even worse than this. Here are a few things that can exacerbate the problems associated with a release.

- When working on a new application, the first deployment to staging is likely to be the most troublesome.
- The longer the release cycle, the longer the development team has to make incorrect assumptions before the deployment occurs, and the longer it will take to fix them.
- In large organizations where the delivery process is divided between different groups such as development, DBA, operations, testing, etc., the cost of coordination between these silos can be enormous, sometimes stalling the release process in ticketing hell. In this scenario, developers, testers, and operations personnel are constantly raising tickets (or sending emails) to

each other to perform any given deployment—and worse, to resolve problems that arise during deployment.

- The bigger the difference between development and production environments, the less realistic are the assumptions that have to be made during development. This can be difficult to quantify, but it's a good bet that if you're developing on a Windows machine and deploying to a Solaris cluster, you are in for some surprises.
- If your application is installed by users or contains components that are, you may not have much control over their environments, especially outside of a corporate setting. In this case, a great deal of extra testing will be required.

Instead . . .

The remedy is to integrate the testing, deployment, and release activities into the development process. Make them a normal and ongoing part of development so that by the time you are ready to release your system into production there is little to no risk, because you have rehearsed it on many different occasions in a progressively more production-like sequence of test environments. Make sure everybody involved in the software delivery process, from the build and release team to testers to developers, work together from the start of the project.

We are test addicts, and the extensive use of continuous integration and continuous deployment, as a means of testing both our software and our deployment process, is a cornerstone of the approach that we describe.

Antipattern: Manual Configuration Management of Production Environments

Many organizations manage the configuration of their production environments through a team of operations people. If a change is needed, such as a change to database connection setting or an increase in the number of threads in a thread pool on an application server, then it is carried out manually on the production servers. If a record is kept of such a change, it is probably an entry in a change management database.

Signs of this antipattern are:

- Having deployed successfully many times to staging, the deployment into production fails.
- Different members of a cluster behave differently—for example, one node sustaining less load or taking longer to process requests than another.
- The operations team take a long time to prepare an environment for a release.

- You cannot step back to an earlier configuration of your system, which may include operating system, application server, web server, RDBMS, or other infrastructural settings.
- Servers in clusters have, unintentionally, different versions of operating systems, third-party infrastructure, libraries, or patch levels.
- Configuration of the system is carried out by modifying the configuration directly on production systems.

Instead . . .

All aspects of each of your testing, staging, and production environments, specifically the configuration of any third-party elements of your system, should be applied from version control through an automated process.

One of the key practices that we describe in this book is configuration management, part of which means being able to repeatably re-create every piece of infrastructure used by your application. That means operating systems, patch levels, OS configuration, your application stack, its configuration, infrastructure configuration, and so forth should all be managed. You should be able to re-create your production environment exactly, preferably in an automated fashion. Virtualization can help you get started with this.

You should know exactly what is in production. That means that every change made to production should be recorded and auditable. Often, deployments fail because somebody patched the production environment last time they deployed, but the change was not recorded. Indeed it should not be possible to make manual changes to testing, staging, and production environments. The only way to make changes to these environments should be through an automated process.

Applications often depend on other applications. It should be possible to see at a glance exactly what the currently released version of every piece of software is.

While releases can be exhilarating, they can also be exhausting and depressing. Almost every release involves last-minute changes, such as fixing the database login details or updating the URL for an external service. There should be a way of introducing such changes so that they are both recorded and tested. Again, automation is essential. Changes should be made in version control and then propagated to production through an automated process.

It should be possible to use the same automated process to roll back to a previous version of production if the deployment goes wrong.

Can We Do Better?

You bet, and the goal of this book is to describe how. The principles, practices, and techniques we describe are aimed at making releases boring, even in complex “enterprise” environments. Software release can—and should—be a low-risk, frequent, cheap, rapid, and predictable process. These practices have been developed over the last few years, and we have seen them make a huge difference

in many projects. All of the practices in this book have been tested in large enterprise projects with distributed teams as well as in small development groups. We know that they work, and we know that they scale to large projects.

The Power of Automated Deployment

One of our clients used to have a large team of people dedicated to each release. The team worked together for seven days, including the entire weekend, to get the application into production. Their success rate was poor, with many releases introducing errors or requiring high levels of intervention on the day of release as well as, often, patches and fixes on subsequent days to correct errors introduced with the release or caused by human errors in configuring the new software.

We helped them to implement a sophisticated automated build, deploy, test, and release system and to introduce the development practices and techniques necessary to support it. The last release we saw took seven seconds to deploy the application into production. No one noticed anything had happened, except of course that the new behaviors that the release implemented suddenly became available. Had the successful deployment of the system behind this major website failed for any reason, we could have backed out the change in the same amount of time.

Our goal is to describe the use of deployment pipelines, combined with high levels of automation of both testing and deployment and comprehensive configuration management to deliver push-button software releases. That is, push-button software releases to any deployment target—development, test, or production.

Along the way we will describe the pattern itself and the techniques that you will need to adopt to make it work. We will provide advice on different approaches to solving some of the problems that you will face. We have found that the advantages of such an approach vastly outweigh the costs of achieving it.

None of this is outside the reach of any project team. It does not require rigid process, significant documentation, or lots of people. By the end of this chapter, we hope that you will understand the principles behind this approach.

How Do We Achieve Our Goal?

As we said, our goal as software professionals is to deliver useful, working software to users as quickly as possible.

Speed is essential because there is an opportunity cost associated with not delivering software. You can only start to get a return on your investment once your software is released. So, one of our two overriding goals in this book is to find ways to reduce *cycle time*, the time it takes from deciding to make a change, whether a bugfix or a feature, to having it available to users.

Delivering fast is also important because it allows you to verify whether your features and bugfixes really are useful. The decision maker behind the creation of an application, who we'll call the customer, makes hypotheses about which features and bugfixes will be useful to users. However, until they are in the hands of users who vote by choosing to use the software, they remain hypotheses. It is therefore vital to minimize cycle time so that an effective feedback loop can be established.

An important part of usefulness is quality. Our software should be fit for its purpose. Quality does not equal perfection—as Voltaire said, “The perfect is the enemy of the good,”—but our goal should always be to deliver software of sufficient quality to bring value to its users. So while it is important to deliver our software as quickly as possible, it is essential to maintain an appropriate level of quality.

So, to slightly refine our goal, we want to find ways to deliver high-quality, valuable software in an efficient, fast, and reliable manner.

We, and our fellow practitioners, have discovered that in order to achieve these goals—low cycle time and high quality—we need to make frequent, automated releases of our software. Why is this?

- **Automated.** If the build, deploy, test, and release process is not automated, it is not repeatable. Every time it is done, it will be different, because of changes in the software, the configuration of the system, the environments, and the release process. Since the steps are manual, they are error-prone, and there is no way to review exactly what was done. This means there is no way to gain control over the release process, and hence to ensure high quality. Releasing software is too often an art; it should be an engineering discipline.
- **Frequent.** If releases are frequent, the delta between releases will be small. This significantly reduces the risk associated with releasing and makes it much easier to roll back. Frequent releases also lead to faster feedback—indeed, they require it. Much of this book concentrates on getting feedback on changes to your application and its associated configuration (including its environment, deployment process, and data) as quickly as possible.

Feedback is essential to frequent, automated releases. There are three criteria for feedback to be useful.

- Any change, of whatever kind, needs to trigger the feedback process.
- The feedback must be delivered as soon as possible.
- The delivery team must receive feedback and then act on it.

Let's examine these three criteria in detail and consider how we can achieve them.

Every Change Should Trigger the Feedback Process

A working software application can be usefully decomposed into four components: executable code, configuration, host environment, and data. If any of them changes, it can lead to a change in the behavior of the application. Therefore we need to keep all four of these components under control and ensure that a change in any one of them is verified.

Executable code changes when a change is made to the source code. Every time a change is made to the source code, the resulting binary must be built and tested. In order to gain control over this process, building and testing the binary should be automated. The practice of building and testing your application on every check-in is known as continuous integration; we describe it in detail in Chapter 3.

This executable code should be the same executable code that is deployed into every environment, whether it is a testing environment or a production environment. If your system uses a compiled language, you should ensure that the binary output of your build process—the executable code—is reused everywhere it is needed and never rebuilt.

Anything that changes between environments should be captured as configuration information. Any change to an application's configuration, in whichever environment, should be tested. If the software is to be installed by the users, the possible configuration options should be tested across a representative range of example systems. Configuration management is discussed in Chapter 2.

If the environments the application is to be deployed into change, the whole system should be tested with the changes to the environment. This includes changes in the operating system configuration, the software stack that supports the application, the network configuration, and any infrastructure and external systems. Chapter 11 deals with managing infrastructure and environments, including automation of the creation and maintenance of testing and production environments.

Finally, if the structure of the data changes, this change must also be tested. We discuss data management in Chapter 12.

What is the feedback process? It involves testing every change in a fully automated fashion, as far as possible. The tests will vary depending on the system, but they will usually include at least the following checks.

- The process of creating the executable code must work. This verifies that the syntax of your source code is valid.
- The software's unit tests must pass. This checks that your application's code behaves as expected.

- The software should fulfill certain quality criteria such as test coverage and other technology-specific metrics.
- The software's functional acceptance tests must pass. This checks that your application conforms to its business acceptance criteria—that it delivers the business value that was intended.
- The software's nonfunctional tests must pass. This checks that the application performs sufficiently well in terms of capacity, availability, security, and so on to meet its users' needs.
- The software must go through exploratory testing and a demonstration to the customer and a selection of users. This is typically done from a manual testing environment. In this part of the process, the product owner might decide that there are missing features, or we might find bugs that require fixing and automated tests that need creating to prevent regressions.

The environments these tests run in must be as similar as possible to production, to verify that any changes to our environments have not affected the application's ability to work.

The Feedback Must Be Received as Soon as Possible

The key to fast feedback is automation. With fully automated processes, your only constraint is the amount of hardware that you are able to throw at the problem. If you have manual processes, you are dependent on people to get the job done. People take longer, they introduce errors, and they are not auditable. Moreover, performing manual build, test, and deployment processes is boring and repetitive—far from the best use of people. People are expensive and valuable, and they should be focused on producing software that delights its users and then delivering those delights as fast as possible—not on boring, error-prone tasks like regression testing, virtual server provisioning, and deployment, which are best done by machines.

However, implementing a deployment pipeline is resource-intensive, especially once you have a comprehensive automated test suite. One of its key objectives is to optimize for human resource usage: We want to free people to do the interesting work and leave repetition to machines.

We can characterize the tests in the commit stage of the pipeline (Figure 1.1) as follows.

- They run fast.
- They are as comprehensive as possible—that is to say, they cover more than 75% or so of the codebase, so that when they pass, we have a good level of confidence that the application works.

- If any of them fails, it means our application has a critical fault and should not be released under any circumstances. That means that a test to check the color of a UI element should not be included in this set of tests.
- They are as environment-neutral as possible—that is, the environment does not have to be an exact replica of production, which means it can be simpler and cheaper.

On the other hand, the tests in the later stages have the following general characteristics.

- They run more slowly and therefore are candidates for parallelization.
- Some of them may fail, and we may still choose to release the application under some circumstances (perhaps there is a critical fix in the release candidate that causes the performance to drop below a predefined threshold—but we might make the decision to release anyway).
- They should run on an environment that is as similar as possible to production, so in addition to the direct focus of the test they also test the deployment process and any changes to the production environment.

This organization of the testing process means that we have a high level of confidence in the software after the first set of tests, which run fastest on the cheapest hardware. If these tests fail, the release candidate does not progress to later stages. This ensures optimal use of resources. There is much more on pipelining in Chapter 5, “Anatomy of the Deployment Pipeline,” and the later Chapters 7, 8, and 9 which describe the commit testing stage, automated acceptance testing, and testing nonfunctional requirements.

One of the fundamentals of our approach is the need for fast feedback. Ensuring fast feedback on changes requires us to pay attention to the process of developing software—in particular, to how we use version control and how we organize our code. Developers should commit changes to their version control system frequently, and split code into separate components as a way of managing large or distributed teams. Branching should, in most circumstances, be avoided. We discuss incremental delivery and the use of components in Chapter 13, “Managing Components and Dependencies,” and branching and merging in Chapter 14, “Advanced Version Control.”

The Delivery Team Must Receive Feedback and Then Act on It

It is essential that everybody involved in the process of delivering software is involved in the feedback process. That includes developers, testers, operations staff, database administrators, infrastructure specialists, and managers. If people in